

Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	18 June 2026
Name	Dhanish Ladwani
Project Name	ShopEZ - E-Commerce Application
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per Table-1 & Table-2.

ShopEZ - 3-Tier MERN-based E-Commerce Architecture

ShopEZ – 3-Tier Architecture

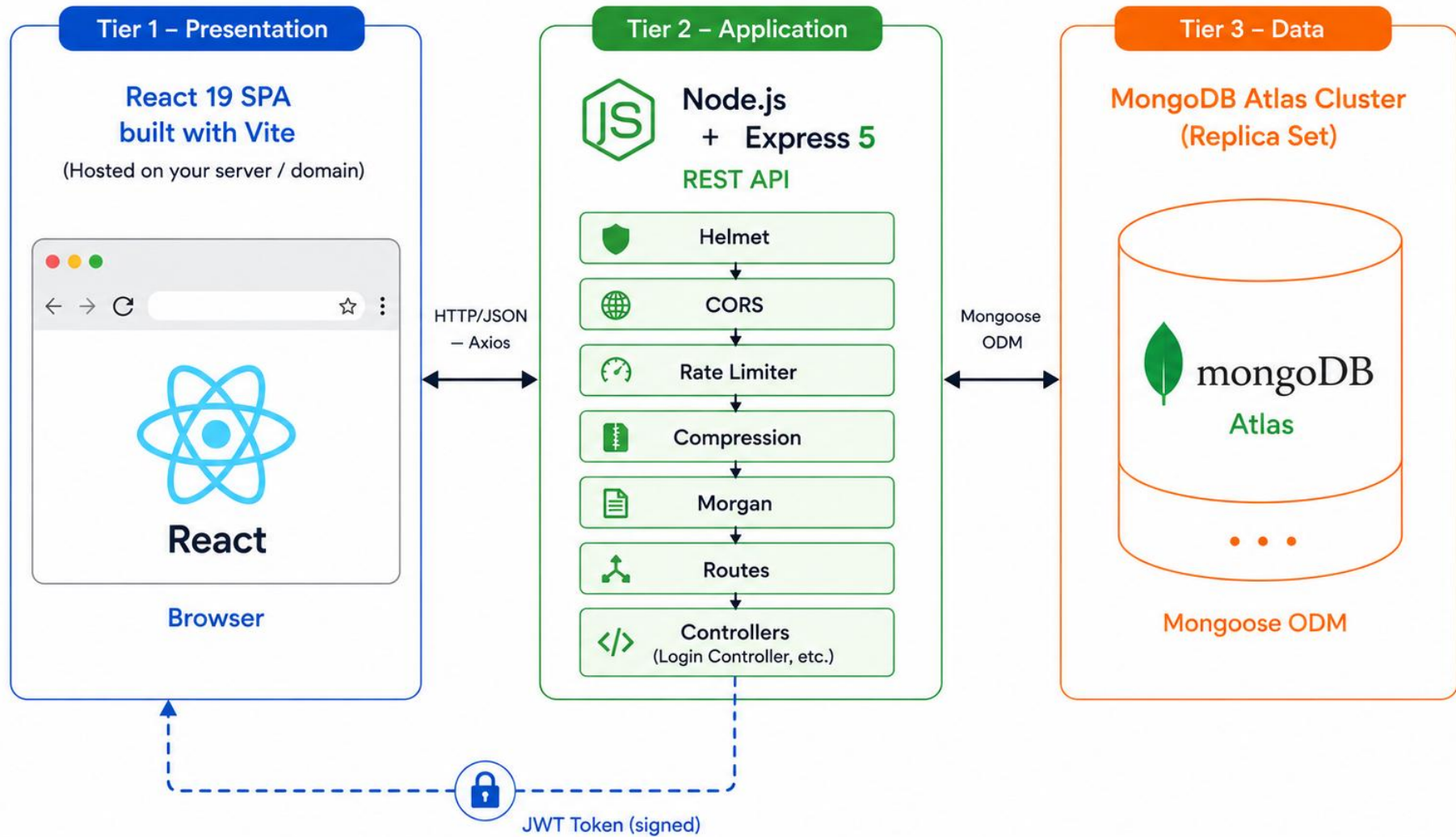


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	React 19 SPA served by Vite. Customers interact via product catalog, cart, checkout, and order history pages. Admins interact via a dedicated dashboard with analytics, product management, order management, and user management pages.	React 19, React Router DOM v7, React Icons, Vite 8, Vanilla CSS
2	Application Logic – Frontend	Client-side state management using React Context API (AuthContext, CartContext). Axios handles all HTTP requests to the REST API. React Router DOM manages client-side routing with protected routes for authenticated users and admin.	React Context API, Axios v1, React Router DOM v7
3	Application Logic – Backend	RESTful API built with Express 5. Implements MVC architecture with separate routes, controllers, and models. Handles user auth, product CRUD, cart operations, order processing, and admin analytics.	Node.js, Express 5, ES Modules (type: module)
4	Authentication & Security	JWT-based stateless authentication. Passwords hashed with bcryptjs. Helmet secures HTTP response headers. express-rate-limit restricts auth routes to 30 req/15 min and API routes to 300 req/15 min. CORS configured for permitted frontend origin only.	jsonwebtoken v9, bcryptjs v3, Helmet v8, express-rate-limit v8, cors v2
5	Database	MongoDB document database. Mongoose ODM manages schemas and models for Users, Products, Orders, and Carts. Indexed fields on product title, category, and gender for efficient filtering and search.	MongoDB, Mongoose v9
6	Cloud Database	MongoDB Atlas M0 shared cluster (free tier) hosted on the cloud. Provides managed backups, monitoring, and high availability with replica sets.	MongoDB Atlas
7	File Storage	Product images are stored as URL strings (external image URLs) in the MongoDB Products collection. No binary file upload is required; images are referenced by URL in the product schema.	URL-based image references (no local file storage)
8	Environment Configuration	Environment variables (MongoDB URI, JWT secret, PORT, allowed origin) managed via dotenv. Configuration is kept separate from application code using a .env file.	dotenv v17
9	Logging & Monitoring	HTTP request logging in development mode using Morgan middleware. Provides method, URL, status code, and response time for every incoming request.	Morgan v1

S.No	Component	Description	Technology
10	Infrastructure (Server / Cloud)	Development: Local Node.js server (nodemon for hot-reload) + Vite dev server. Production: Backend deployable to any Node.js cloud host (Render, Railway, Heroku). Frontend deployable as static site (Vercel, Netlify, or any CDN).	Node.js runtime, nodemon v3 (dev) Vite build (production static), Render / Vercel (deployment)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frontend: React 19 (UI library), React Router DOM v7 (routing), Axios v1 (HTTP client), Vite 8 (build tool), React Icons v5. Backend: Express 5 (web framework), Mongoose v9 (ODM), bcryptjs v3 (hashing), jsonwebtoken v9, Helmet v8, compression v1, cors v2, morgan v1, express-rate-limit v8.	React, Vite, React Router DOM, Axios (MIT licensed) Express, Mongoose, bcryptjs, JWT, Helmet, compression (MIT / ISC licensed)
2	Security Implementations	JWT (HS256) for stateless authentication; tokens verified on every protected API request. bcryptjs password hashing (salt rounds: 10) — plaintext passwords are never stored. Helmet v8 sets secure HTTP headers (X-Content-Type-Options, X-Frame-Options, HSTS, etc.). express-rate-limit: auth routes 30 req/15 min, general API 300 req/15 min — prevents brute-force. CORS restricted to the configured frontend origin only. Admin-only routes protected by a dedicated isAdmin middleware.	jsonwebtoken v9 (HS256) bcryptjs v3 Helmet v8 express-rate-limit v8 cors v2 Custom isAdmin middleware
3	Scalable Architecture	3-Tier Architecture: Presentation (React SPA) → Application (Express REST API) → Data (MongoDB Atlas). The backend is fully stateless (JWT-based), allowing multiple API server instances to run behind a load balancer without shared session state. MongoDB Atlas supports vertical and horizontal scaling (sharding) as data grows. Frontend is a static SPA deployable on any CDN independently of the backend.	React SPA (Tier 1) Express REST API — stateless (Tier 2) MongoDB Atlas — scalable cluster (Tier 3)

S.No	Characteristics	Description	Technology
4	Availability	MongoDB Atlas M0+ clusters run with replica sets, providing automatic failover and redundancy. The stateless REST API can be deployed as multiple instances behind a cloud load balancer (e.g., Render auto-scaling, AWS ALB). The React frontend, once built, is served as static assets from a CDN (e.g., Vercel Edge Network) with near-100% availability.	MongoDB Atlas replica sets Cloud load balancer (Render / AWS) Vercel / Netlify CDN (frontend)
5	Performance	Response compression: the compression middleware (gzip/deflate) reduces API payload size on all responses. Vite build: tree-shaking and code-splitting produce optimised production bundles with minimal JavaScript. Rate limiting reduces server load from abusive clients. React Context avoids unnecessary prop drilling; component re-renders are scoped to relevant state changes. MongoDB indexed fields (title, category, gender) speed up product search and filter queries.	compression v1 (gzip) Vite 8 (tree-shaking / code-split) express-rate-limit v8 React Context API MongoDB indexes (Mongoose)

References:

<https://c4model.com/>

<https://reactjs.org/>

<https://expressjs.com/>

<https://mongoosejs.com/>

<https://vitejs.dev/>

<https://www.mongodb.com/atlas>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>